

MECHATRONIC SYSTEM INTEGRATION
MCTA 3203

EXPERIMENT 2 :
SERIAL COMMUNICATION BETWEEN ARDUINO & PYTHON

SECTION 2
SEMESTER 1, 2026/2027

INSTRUCTOR :
DR. AHMAD IMRAN BIN IBRAHIM
DR. ING. WAHJU SEDIONO
TS. DR. YASIR BIN MOHD MUSTAFAH

DATE OF SUBMISSION : 17 OCTOBER 2026

GROUP 9

NO.	NAME	NO. MATRIC
1	MUHAMMAD DANISH FITRI BIN MOHD SHARIFUDDIN	2413559
2	MUHAMMAD IZZ ZIKRY BIN AZWARI	2218899
3	SAMER Y. I. LUBBAD	2213959
4	MUHAMMAD FARIS IZHAN BIN JOHARI	2413501

ABSTRACT

The main objective of this experiment was to establish bidirectional serial communication between a microcontroller and a computer using Python. The hardware setup utilized an Arduino Uno, a rotary potentiometer, a standard LED with a 220-ohm resistor, and a servo motor. During the procedure, the Arduino was programmed to read analog sensor data from the potentiometer and send it to the computer via a USB COM 8 port. A Python script, utilizing the 'pyserial' and 'matplotlib' libraries, was used to read this data, display it on a scrolling real-time graph, and send command signals back to the Arduino. The system successfully turned on the LED when the sensor value crossed a software threshold of 511. Furthermore, the servo motor successfully mirrored the potentiometer's rotation. A keyboard interrupt feature was also implemented, allowing the servo to be paused and resumed using specific key presses on the computer. The main technical challenges encountered were COM port access errors and mechanical servo jitter, which were resolved by properly managing software monitors and adding a deadband to the code. Upgrading to an external power supply is recommended for future projects to improve motor stability.

TABLE OF CONTENTS

1.0 INTRODUCTION....4
1.1 MATERIALS & EQUIPMENTS....4
1.2 EXPERIMENTAL SETUP....5
1.2.1 Circuit Setup....5
1.2.2 Circuit diagram....5
1.3 METHODOLOGY & PROGRAMMING LOGIC....6
1.3.1 Experiment steps....6
1.3.2 Arduino Code....6
1.3.2 Python Code....8
1.4 DATA COLLECTION & ANALYSIS....10
1.4.1 Data Collection....10
1.4.2 Data Analysis....10
1.5 RESULTS & DISCUSSION....11
1.5.1 Results....11
1.5.2 Discussion....11
1.6 CONCLUSION....12
1.7 RECOMMENDATIONS....12
REFERENCES....13
APPENDICES....13
ACKNOWLEDGEMENT....13
STUDENTS DECLARATION....14
Certificate of Originality and Authenticity....14

1.0 INTRODUCTION

Serial communication is a fundamental concept in modern mechatronics. It allows simple microcontrollers to talk to more powerful computers. This is highly useful because microcontrollers like the Arduino are great at interacting with physical sensors and motors, while computers are much better at processing complex logic and displaying graphical interfaces.

In this experiment, an Arduino Uno is linked to a computer to create a complete hardware-software loop. A rotary potentiometer acts as the physical input, and a servo motor and LED act as the physical outputs. Instead of the Arduino doing all the thinking, the computer runs a Python script to process the data and make the decisions.

By sending data back and forth over a USB cable, this setup demonstrates how to plot live sensor data on a computer screen and how to control physical actuators using a computer keyboard. This method is widely used in real-world engineering to monitor and control industrial machines.

1.1 MATERIALS & EQUIPMENTS

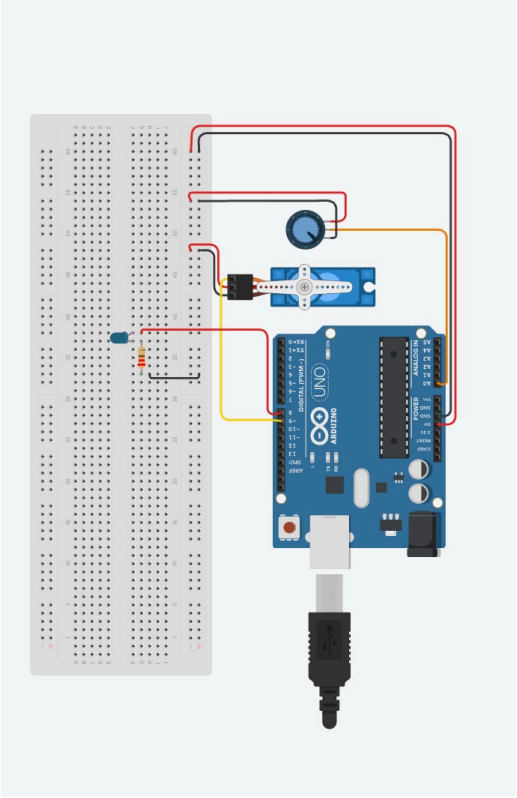
1. Arduino Uno board (1)
2. Rotary Potentiometer (1)
3. Standard 5mm LED (1)
4. Servo Motor (1)
5. 220-ohm resistor (1)
6. Breadboard (1)
7. USB connection cable (1)
8. Jumper wires
9. Laptop with PyCharm and Arduino IDE installed

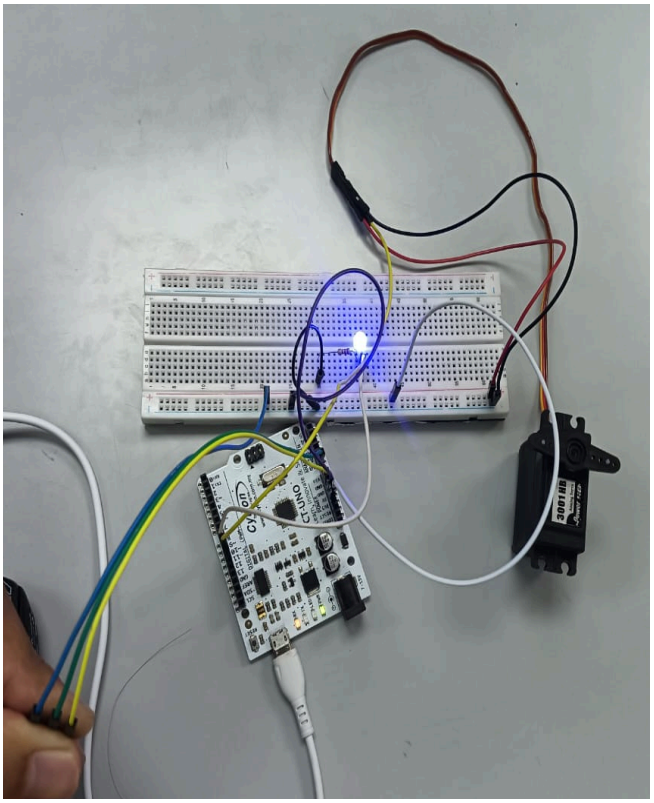
1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup

1. The rotary potentiometer was connected to the Arduino Uno. One outer pin was connected to the 5V supply, the other outer pin to GND, and the center signal pin to the analog input A0.
2. The LED was set up as a status indicator. The long leg (anode) was connected to digital pin 8. The short leg (cathode) was connected to a 220-ohm current-limiting resistor, which was then routed to GND.
3. The servo motor was connected to the microcontroller. The signal wire (usually yellow or orange) was connected to the PWM-capable digital pin 9. The power wire (red) was connected to 5V, and the ground wire (black or brown) was connected to GND.

1.2.2 Circuit diagram





1.3 METHODOLOGY & PROGRAMMING LOGIC

1.3.1 Experiment steps

1. The electronic components were wired on the breadboard according to the circuit setup.
2. The C++ code was uploaded to the Arduino Uno to handle reading the sensor and moving the motor.
3. The Arduino IDE Serial Monitor was completely closed to prevent 'Access Denied' errors on the COM port.
4. A Python script was executed in PyCharm to open the COM port and start the real-time Matplotlib graph.
5. The potentiometer was rotated manually to test if the graph updated and if the servo motor moved accordingly.
6. The LED was observed to ensure it turned on only when the analog value crossed the halfway point.
7. The 'h' key was pressed on the computer keyboard to pause the motor, and the 'r' key was pressed to resume it.

1.3.2 Arduino Code

```
#include <Servo.h>



Servo myservo;

const int potPin = A0;

const int ledPin = 8;

bool isHalted = false; // Tracks if the servo should be paused



void setup() {

  Serial.begin(9600);

  myservo.attach(9);

  pinMode(ledPin, OUTPUT);
```

```
}


void loop() {
  // 1. Check for incoming commands from Python
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'h') {
      isHalted = true;  // Halt servo movement
    } else if (cmd == 'r') {
      isHalted = false; // Resume servo movement
    } else if (cmd == '1') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    } else if (cmd == '0') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }

  // 2. Read the potentiometer and send data to Python
  int potValue = analogRead(potPin);
  Serial.println(potValue);

  // 3. Move the servo ONLY if it is not halted
  if (!isHalted) {
    int angle = map(potValue, 0, 1023, 5, 170); // Convert sensor range
to angle
```

```
    myservo.write(angle);

  }



  delay(100); // Small delay for stable plotting

}
```

1.3.2 Python Code

```
import serial
import matplotlib.pyplot as plt
import time

# Connect to your active port
ser = serial.Serial('COM8', 9600, timeout=0.5)
time.sleep(2)

# Set up Matplotlib interactive plotting
plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []
time_step = 0  # Independent counter for the X-axis so the graph scrolls


# Function to catch keyboard presses while the graph is open
def on_press(event):
   if event.key == 'h':
       ser.write(b'h')
       print(">>> HALT COMMAND SENT: Servo paused. (Press 'r' to resume)")
   elif event.key == 'r':
       ser.write(b'r')
       print(">>> RESUME COMMAND SENT: Servo active.")


# Connect the keyboard listener to the graph window
fig.canvas.mpl_connect('key_press_event', on_press)

try:
   print("Starting real-time plot. Click on the graph window and press 'h' to
halt the servo.")
   while True:
       if ser.in_waiting > 0:
           pot_value_str = ser.readline().decode().strip()

           if pot_value_str.isdigit():
               pot_value = int(pot_value_str)
               print(f"Potentiometer Value: {pot_value}")
```

```
                # Turn on LED if pot crosses midway point (511)
                if pot_value > 511:
                    ser.write(b'1')
                else:
                    ser.write(b'0')

                # Update plotting lists using the independent counter
                time_step += 1
                x_vals.append(time_step)
                y_vals.append(pot_value)

                # Keep the graph from getting too crowded (show last 50
points)
                if len(x_vals) > 50:
                    x_vals = x_vals[-50:]
                    y_vals = y_vals[-50:]

                # Refresh the graph
                ax.clear()
                ax.plot(x_vals, y_vals, color='blue')
                ax.set_ylim(0, 1023)
                ax.set_title("Real-Time Potentiometer Data")
                ax.set_ylabel("Analog Value (0-1023)")
                plt.pause(0.01)

except KeyboardInterrupt:
   pass
finally:
   ser.close()
   plt.ioff()
   plt.show()
   print("Serial connection closed.")
```

1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection

Observation	Potentiometer Reading (0-1023)	Python Logic Evaluation	Hardware Status (LED)	Hardware Status (Servo Angle)
1	0 (Minimum)	Value <= 511	OFF	0°
2	256	Value <= 511	OFF	~45°
3	511 (Threshold)	Value <= 511	OFF	90°
4	512 (Threshold Crossed)	Value > 511	ON	90°
5	1023 (Maximum)	Value > 511	ON	180° (or software limit)
6	Keyboard 'h' Pressed	Interrupt Triggered	Retains current state	Halts at current angle

1.4.2 Data Analysis

The collected data confirmed the following points:

1. The bidirectional serial connection was stable. Analog data traveled to the computer, and byte commands ('1', '0', 'h', 'r') traveled back to the Arduino.
2. The Python script successfully triggered the LED exactly at the halfway mark of the analog range (511).
3. The software angle limit (5 to 170 degrees) successfully stopped the motor from hitting its mechanical boundaries, preventing power crashes.

1.5 RESULTS & DISCUSSION

1.5.1 Results

The final task successfully demonstrated a complete hardware-software integration loop.

1. The Matplotlib library generated a smooth, scrolling line graph that updated in real-time without crashing.
2. The LED responded instantly to the Python software logic.
3. The servo motor tracked the potentiometer position accurately.
4. The keyboard interrupt receiver captured key presses correctly, successfully pausing and resuming the physical motor.

1.5.2 Discussion

1. Interpretation of Results: The experiment proved that complex logic can be handled by a computer rather than the microcontroller itself. This approach makes adding user interfaces and visual graphs much easier.
2. Discrepancies Between Expected and Observed Outcomes: Initially, the servo motor experienced severe jittering. It vibrated aggressively instead of staying still. This was fixed by writing a "deadband" in the Arduino code so the motor only moved if the sensor value changed significantly.
3. Sources of Error and Improvement:
    - COM Port Conflicts: A major source of error was the "PermissionError: Access is denied" warning in Python. This happened because the Windows operating system only allows one program to use the COM port. The issue was solved by strictly closing the Arduino IDE monitors before running PyCharm.
    - Power Limitations: The servo drew too much current from the laptop USB port when turning to extreme angles. Shrinking the mapped range to a safe 5 to 170 degrees improved the physical stability of the system.

1.6 CONCLUSION

This laboratory project successfully achieved its goal of setting up two-way serial communication between an Arduino and a Python environment. By building the circuit and writing the specific codes, a system was created where sensor data could be plotted live on a screen, and a motor could be controlled by a computer keyboard.

During the session, practical knowledge was gained regarding data processing, software delays, and managing communication ports. Real-world hardware problems, such as electrical noise causing servo jitter, were successfully solved using software filters like deadbands.

For the Islamic reflection, this lab session highlights the importance of seeking knowledge and understanding systems. Building electronic circuits requires strict protocols and careful communication between different parts. When studying how digital logic and physical actuators work, the laws of physics are being observed. These rules are part of the perfect order that Allah established in the universe. Mastering these engineering concepts allows students to build beneficial technologies to serve society.

1.7 RECOMMENDATIONS

1. Add an External Power Supply:

- Standard servo motors draw large amounts of current. Powering the servo directly from the Arduino 5V pin can cause the laptop USB port to shut down. Using a dedicated external battery pack for the motor would make the system much more stable.

1. Increase the Baud Rate:

- The current communication speed is set to 9600 baud. Increasing this to 115200 would make the data transfer much faster. This would reduce any visual lag between turning the physical knob and seeing the graph update on the screen.

REFERENCES

Arduino. (2026). *Arduino language reference*. https://www.arduino.cc/reference/en/

Department of Mechatronics Engineering. (2026). *Laboratory manual for mechatronics system integration*. International Islamic University Malaysia.

Floyd, T. L. (2015). *Digital fundamentals* (11th ed.). Pearson.

Margolis, M., Jepson, B., & Weldin, N. R. (2020). *Arduino cookbook: Recipes to begin, expand, and enhance your projects* (3rd ed.). O'Reilly Media.

APPENDICES

Video on task1 part 1: https://youtube.com/shorts/ZSlXDijx4m8

Video on task1 part 2: https://youtu.be/d7UxXTifwlo

Video on task 2 : https://youtu.be/Ve5jDhu4A2A

ACKNOWLEDGEMENT

First and foremost, a sincere thank you goes to the lecturer for the constant guidance during this lab session. The clear explanations made connecting the Arduino code to the physical breadboard much easier to figure out.

Appreciation is also given to the Mechatronics Engineering department at IIUM for providing a great lab environment. Having access to all the right microcontrollers and electronic parts was a huge help in getting the digital counter to work properly.

A special mention goes to the lab technicians as well. Their quick help with checking the hardware and replacing faulty jumper wires saved a lot of time and kept the experiment moving.

Finally, a huge thanks goes out to the lab partners and classmates. Spending hours troubleshooting the circuit, sharing ideas, and fixing coding errors together made this a really good learning experience. The great teamwork is what made this whole project a success.

STUDENTS DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature :  Read [/]

Name : Muhammad Danish Fitri Bin Mohd Sharifuddin Understand [/]

Matric No : 2413559 Agree [/]

Signature :  Read [/]

Name : Muhammad Izz Zikry Bin Azwari Understand [/]

Matric No : 2218899 Agree [/]

Signature : 

Read [/]

Name : Samer Y. I. Lubbad

Understand [/]

Matric No : 2213959

Agree [/]

Signature : 

Read [/]

Name : Muhammad Faris Izhan Bin Johari

Understand [/]

Matric No : 2413501

Agree [/]